Manual Técnico de Despliegue — SciCalc Pro

Versión: 1.0

Fecha: Marzo 2026

Audiencia: Desarrolladores, DevOps, administradores de sistemas

---

Tabla de Contenidos

1. Prerrequisitos
2. Instalación y configuración
3. Variables de entorno
4. Ejecutar el servidor
5. Ejecución de pruebas unitarias
6. Ejecución de pruebas de integración
7. Ejecución de pruebas E2E con Selenium
8. Análisis de calidad con SonarQube
9. Pipeline CI/CD con GitHub Actions
10. Endpoints de la API
11. Arquitectura del sistema

---

1. Prerrequisitos

Antes de instalar y ejecutar SciCalc Pro, asegúrese de contar con el siguiente software instalado en el sistema:

Software	Versión mínima	Propósito
Python	3.12	Ejecución del backend FastAPI
pip	23+	Gestor de paquetes Python
Docker	24+	Contenerización de servicios (PostgreSQL, SonarQube)
Docker Compose	2.20+	Orquestación de contenedores
PostgreSQL	15	Base de datos relacional (o vía Docker)
Google Chrome	Estable más reciente	Requerido para pruebas E2E con Selenium
Git	2.40+	Control de versiones

> **Nota para Windows:** El módulo `asyncpg` requiere `SelectorEventLoop` en Windows. El archivo `conftest.py` de pruebas E2E ya aplica este parche automáticamente para Python 3.12+.

---

2. Instalación y configuración

2.1 Clonar el repositorio

```
git clone <https://github.com/><org>/SciCalcPro.git
cd SciCalcPro
```

2.2 Crear y activar entorno virtual

```
# Linux / macOS
python3.12 -m venv .venv
source .venv/bin/activate

# Windows
python -m venv .venv
.venv\\Scripts\\activate
```

2.3 Instalar dependencias del backend

```
cd backend
pip install -e ".[dev]"
```

El archivo `pyproject.toml` incluye las dependencias de producción y desarrollo (pytest, ruff, coverage, selenium, webdriver-manager, httpx, etc.).

2.4 Configurar variables de entorno

Cree el archivo `.env` dentro de `backend/`:

```
cp .env.example .env
# Edite .env con los valores reales (ver sección 3)
```

2.5 Iniciar servicios con Docker Compose

```
# Desde la raíz del repositorio
docker compose up -d
```

Esto levanta:

- **PostgreSQL 15** en el puerto `5432`.
- **SonarQube Community** en el puerto `9000` (opcional para análisis local).

2.6 Aplicar migraciones de base de datos

```
cd backend
alembic upgrade head
```

Este comando crea todas las tablas definidas en los modelos SQLAlchemy (users, profiles, calculation_history, user_preferences, favorite_conversions, sessions, sync_log).

---

3. Variables de entorno

Todas las variables de entorno se definen en `backend/.env`. **Nunca** commitee este archivo al repositorio.

Variable	Descripción	Valor de ejemplo
`DATABASE_URL`	URL de conexión a PostgreSQL con driver asyncpg	`postgresql+asyncpg://scicalc:secret@localhost:5432/scicalc_db`
`SECRET_KEY`	Clave secreta para firma de tokens JWT (mínimo 32 caracteres)	`clave-super-secreta-de-al-menos-32-chars!`
`ALGORITHM`	Algoritmo de firma JWT	`HS256`
`ACCESS_TOKEN_EXPIRE_MINUTES`	Duración del token de acceso en minutos	`15`
`REFRESH_TOKEN_EXPIRE_DAYS`	Duración del token de refresco en días	`7`

> **Seguridad:** Use una `SECRET_KEY` de al menos 32 caracteres aleatorios. Genérela con: `python -c "import secrets; print(secrets.token_hex(32))"`.

---

4. Ejecutar el servidor

4.1 Inicio del servidor de desarrollo

```
cd backend
python run.py
```

El servidor queda disponible en: `http://localhost:8000`

Rutas principales:

- `http://localhost:8000/docs` — Documentación Swagger UI
- `http://localhost:8000/redoc` — Documentación ReDoc
- `http://localhost:8000/health` — Health check
- `http://localhost:8000/web/login` — Interfaz web de login
- `http://localhost:8000/web/register` — Interfaz web de registro
- `http://localhost:8000/web/calculator` — Interfaz web de la calculadora

4.2 Inicio con Uvicorn directamente

```
cd backend
uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload
```

El flag `--reload` recarga automáticamente al detectar cambios en el código (solo para desarrollo).

4.3 Producción con Nginx

En producción se recomienda colocar Nginx como reverse proxy frente a Uvicorn:

- Nginx escucha en el puerto `443` (HTTPS) y reenvía a `localhost:8000`.
- El certificado TLS debe estar configurado en Nginx.
- Usar `gunicorn` con workers `uvicorn.workers.UvicornWorker` para mayor estabilidad en producción.

---

5. Ejecución de pruebas unitarias

Las pruebas unitarias no requieren base de datos. Usan mocks para aislar la lógica de negocio.

```
cd backend
pytest tests/unit/ -v \\
  --cov=src \\
  --cov-report=term-missing \\
  --cov-report=xml:coverage.xml
```

Criterio de éxito: Las pruebas unitarias por sí solas generan cobertura aproximada del 42%. La cobertura total del 90% se alcanza al combinar con las pruebas de integración (ver sección 6).

Archivos de salida:

- `backend/coverage.xml` — Reporte de cobertura para SonarQube.

---

6. Ejecución de pruebas de integración

Las pruebas de integración requieren una instancia de PostgreSQL activa. Configure `DATABASE_URL` antes de ejecutarlas.

```
cd backend
pytest tests/unit/ tests/integration/ -v \\
  --cov=src \\
  --cov-report=term-missing \\
  --cov-report=xml:coverage.xml \\
  --cov-fail-under=90
```

Prerrequisito: La base de datos de prueba debe existir y tener las migraciones aplicadas:

```
alembic upgrade head
```

Criterio de éxito: Cobertura combinada mayor o igual al **90%** (`--cov-fail-under=90`). El pipeline CI fallará si no se alcanza este umbral.

---

7. Ejecución de pruebas E2E con Selenium

Las pruebas E2E levantan el servidor FastAPI en un hilo de fondo y controlan Chrome mediante Selenium WebDriver.

7.1 Prerrequisitos

- Google Chrome instalado en el sistema.
- `DATABASE_URL` apuntando a una base de datos PostgreSQL de prueba (puede ser la misma de integración o una dedicada).

- Migraciones aplicadas.

7.2 Comando de ejecución

```
cd backend
pytest tests/e2e/ -v -m e2e
```

7.3 Cobertura de pruebas E2E

ID	Archivo	Descripción
WEB-001	test_web_pages.py	Renderizado de `/web/login` con todos los elementos DOM requeridos
WEB-002	test_web_pages.py	Renderizado de `/web/register` con todos los elementos DOM requeridos
WEB-003	test_web_pages.py	Renderizado de `/web/calculator` con expression, precision, result, history-list
E2E-002a	test_auth_e2e.py	Registro de usuario nuevo → mensaje de éxito
E2E-002b	test_auth_e2e.py	Registro con email duplicado → mensaje de error
E2E-003a	test_auth_e2e.py	Login con credenciales válidas → token en sessionStorage
E2E-003b	test_auth_e2e.py	Login con contraseña incorrecta → sin token
E2E-004a	test_calculator_e2e.py	Aritmética básica `2 + 3 * 4 = 14`
E2E-004b	test_calculator_e2e.py	Trigonometría `sin(90)` en DEG = 1
E2E-004c	test_calculator_e2e.py	Expresión inválida → mensaje de error
E2E-005a	test_calculator_e2e.py	Historial muestra entradas después de calcular
E2E-005b	test_calculator_e2e.py	Borrar entrada de historial → desaparece del DOM

7.4 Notas de ejecución en Windows

En Windows, el event loop de asyncpg requiere `SelectorEventLoop`. El archivo `conftest.py` aplica el parche automáticamente:

```
if sys.platform == "win32":
    asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
```

7.5 ChromeDriver

El paquete `webdriver-manager` descarga automáticamente la versión de ChromeDriver compatible con el Chrome instalado. No es necesario instalar ChromeDriver manualmente.

---

8. Análisis de calidad con SonarQube

8.1 Iniciar SonarQube con Docker

```
docker run -d \\
  --name sonarqube \\
  -p 9000:9000 \\
  sonarqube:community
```

Espere aproximadamente 60 segundos hasta que SonarQube esté disponible en `http://localhost:9000`.

Credenciales por defecto: `admin` / `admin` (cambiarlas en el primer acceso).

8.2 Crear proyecto en SonarQube

1. Acceda a `http://localhost:9000`.
2. Haga clic en **Create Project** → **Manually**.
3. Nombre del proyecto: `scicalcpro`, clave: `scicalcpro`.
4. Genere un **token de análisis** y guárdelo como `SONAR_TOKEN`.

8.3 Configurar sonar-project.properties

El archivo `backend/sonar-project.properties` debe contener:

```
sonar.projectKey=scicalcpro
sonar.projectName=SciCalc Pro
sonar.sources=src
sonar.tests=tests
sonar.python.coverage.reportPaths=coverage.xml
sonar.host.url=http://localhost:9000
```

8.4 Ejecutar el análisis

Genere primero el reporte de cobertura (sección 6), luego ejecute:

```
cd backend
sonar-scanner \\
  -Dsonar.token=$SONAR_TOKEN \\
  -Dsonar.host.url=http://localhost:9000
```

El informe estará disponible en `http://localhost:9000/dashboard?id=scicalcpro`.

---

9. Pipeline CI/CD con GitHub Actions

El pipeline está definido en `.github/workflows/ci.yml` y se activa en:

- Push a las ramas `main` o `develop`.
- Pull Requests hacia `main`.

9.1 Etapas del pipeline

Etapa	Job	Depende de	Descripción	Criterio de éxito
1	`lint`	—	Análisis estático con `ruff` sobre `src/`	Sin errores de lint
2	`unit-tests`	`lint`	Pruebas unitarias con cobertura parcial (~42%)	Todas las pruebas pasan
3	`integration-tests`	`lint`	Pruebas unitarias + integración con PostgreSQL 15	Cobertura >= 90%
4	`e2e-tests`	`integration-tests`	Pruebas E2E con Selenium + Chrome headless	Todas las pruebas pasan
5	`sonarqube`	`integration-tests`	Análisis SonarQube usando cobertura combinada	Condición: `SONAR_HOST_URL` configurado

9.2 Secrets y variables requeridos en GitHub

Configure estos valores en **Settings** → **Secrets and variables** → **Actions** del repositorio:

Nombre	Tipo	Descripción
`SONAR_TOKEN`	Secret	Token de autenticación de SonarQube
`SONAR_HOST_URL`	Variable (vars)	URL del servidor SonarQube (ej. `https://sonar.miempresa.com`)

> **Nota:** El job `sonarqube` se ejecuta condicionalmente: solo si `vars.SONAR_HOST_URL` está configurado. Esto evita fallos en forks que no tienen SonarQube.

9.3 Base de datos en CI

Los jobs `integration-tests` y `e2e-tests` usan el servicio Docker integrado de GitHub Actions para levantar PostgreSQL 15 Alpine:

```
services:
  postgres:
    image: postgres:15-alpine
    env:
      POSTGRES_DB: scicalc_test
      POSTGRES_USER: scicalc
      POSTGRES_PASSWORD: secret
```

Las variables de entorno del job apuntan automáticamente a este servicio:

```
DATABASE_URL: postgresql+asyncpg://scicalc:secret@localhost:5432/scicalc_test
```

9.4 Instalación de Chrome en CI

El job `e2e-tests` instala Google Chrome estable desde el repositorio oficial de Google:

```
wget -q -O - <https://dl.google.com/linux/linux_signing_key.pub> | sudo apt-key
add -
sudo sh -c 'echo "deb [arch=amd64] <http://dl.google.com/linux/chrome/deb/> sta
ble main" >> /etc/apt/sources.list.d/google-chrome.list'
sudo apt-get update -q && sudo apt-get install -y google-chrome-stable
```

---

10. Endpoints de la API

La documentación completa está disponible en `/docs` (Swagger UI) cuando el servidor está en ejecución.

Autenticación

Método	Ruta	Descripción
`POST`	`/api/v1/auth/register`	Registrar nuevo usuario
`POST`	`/api/v1/auth/login`	Iniciar sesión, obtener JWT
`POST`	`/api/v1/auth/refresh`	Renovar token de acceso
`POST`	`/api/v1/auth/logout`	Invalidar sesión

Usuarios

Método	Ruta	Descripción
`GET`	`/api/v1/users/me`	Obtener datos del usuario autenticado
`PUT`	`/api/v1/users/me`	Actualizar perfil
`DELETE`	`/api/v1/users/{user_id}`	Eliminar cuenta (soft delete)

Calculadora

Método	Ruta	Descripción
`POST`	`/api/v1/calculator/calculate`	Evaluar expresión matemática
`GET`	`/api/v1/calculator/history`	Obtener historial del usuario
`DELETE`	`/api/v1/calculator/history/{id}`	Eliminar entrada del historial

Conversión de unidades

Método	Ruta	Descripción
`POST`	`/api/v1/converter/convert`	Convertir entre unidades

Utilidades

Método	Ruta	Descripción
`GET`	`/health`	Health check del servidor

Interfaces web (Jinja2)

Método	Ruta	Descripción
`GET`	`/web/login`	Página de inicio de sesión
`GET`	`/web/register`	Página de registro
`GET`	`/web/calculator`	Interfaz de la calculadora

---

11. Arquitectura del sistema

11.1 Estilo arquitectónico: Hexagonal (Ports & Adapters)

SciCalc Pro sigue una arquitectura hexagonal para separar la lógica de negocio de los adaptadores externos (base de datos, HTTP, etc.).

```
src/
├── domain/          # Entidades, value objects, reglas de negocio puras
├── application/     # Casos de uso, servicios de aplicación
├── infrastructure/  # Adaptadores: repositorios SQLAlchemy, modelos DB
├── api/             # Adaptadores de entrada: routers FastAPI
├── web/             # Adaptadores de entrada: vistas Jinja2
└── main.py          # Punto de entrada, composición de la aplicación
```

11.2 Capas

Capa	Tecnología	Responsabilidad
Presentación web	Jinja2 + HTML/CSS/JS	Renderizado server-side de páginas web
API REST	FastAPI 0.111	Endpoints JSON, validación con Pydantic

Capa	Tecnología	Responsabilidad
Aplicación	Python puro	Casos de uso, orquestación
Dominio	Python puro	Entidades, reglas de negocio
Persistencia	SQLAlchemy 2.0 + asyncpg	ORM asíncrono sobre PostgreSQL
Autenticación	PyJWT + bcrypt	Tokens JWT, hashing de contraseñas
Cálculo	mpmath	Aritmética de precisión arbitraria

11.3 Flujo de una solicitud de cálculo

```
Navegador
  → POST /api/v1/calculator/calculate
    → Router FastAPI (valida JWT, deserializa body)
      → CalculatorService (caso de uso)
        → ExpressionValidator (regex RNF-006)
        → MathEngine (mpmath, evaluación segura)
        → HistoryRepository (SQLAlchemy async → PostgreSQL)
      ← resultado + entrada guardada en historial
    ← JSON { result, expression, precision }
  ← Actualiza DOM (result, history-list)
```

11.4 Seguridad

- **Autenticación:** JWT HS256, tokens de acceso de 15 min + refresh de 7 días.
- **Contraseñas:** bcrypt con factor de coste 12.
- **Validación de expresiones:** Regex lista blanca (RNF-006) antes de evaluar. Solo permite caracteres matemáticos conocidos.
- **Soft delete:** Todas las entidades incluyen `is_deleted` para eliminación lógica sin pérdida de datos.
- **Variables sensibles:** Cargadas desde variables de entorno, nunca hardcodeadas.

Manual Técnico de Despliegue — SciCalc Pro

Versión: 1.0

Fecha: Marzo 2026

Audiencia: Desarrolladores, DevOps, administradores de sistemas

---

Tabla de Contenidos

1. Prerrequisitos
2. Instalación y configuración
3. Variables de entorno
4. Ejecutar el servidor
5. Ejecución de pruebas unitarias
6. Ejecución de pruebas de integración
7. Ejecución de pruebas E2E con Selenium
8. Análisis de calidad con SonarQube

9. Pipeline CI/CD con GitHub Actions
10. Endpoints de la API
11. Arquitectura del sistema

---

1. Prerrequisitos

Antes de instalar y ejecutar SciCalc Pro, asegúrese de contar con el siguiente software instalado en el sistema:

Software	Versión mínima	Propósito
Python	3.12	Ejecución del backend FastAPI
pip	23+	Gestor de paquetes Python
Docker	24+	Contenerización de servicios (PostgreSQL, SonarQube)
Docker Compose	2.20+	Orquestación de contenedores
PostgreSQL	15	Base de datos relacional (o vía Docker)
Google Chrome	Estable más reciente	Requerido para pruebas E2E con Selenium
Git	2.40+	Control de versiones

> **Nota para Windows:** El módulo `asyncpg` requiere `SelectorEventLoop` en Windows. El archivo `conftest.py` de pruebas E2E ya aplica este parche automáticamente para Python 3.12+.

---

2. Instalación y configuración

2.1 Clonar el repositorio

```
git clone <https://github.com/><org>/SciCalcPro.git
cd SciCalcPro
```

2.2 Crear y activar entorno virtual

```
# Linux / macOS
python3.12 -m venv .venv
source .venv/bin/activate

# Windows
python -m venv .venv
.venv\\Scripts\\activate
```

2.3 Instalar dependencias del backend

```
cd backend
pip install -e ".[dev]"
```

El archivo `pyproject.toml` incluye las dependencias de producción y desarrollo (pytest, ruff, coverage, selenium, webdriver-manager, httpx, etc.).

2.4 Configurar variables de entorno

Cree el archivo `.env` dentro de `backend/`:

```
cp .env.example .env
# Edite .env con los valores reales (ver sección 3)
```

2.5 Iniciar servicios con Docker Compose

```
# Desde la raíz del repositorio
docker compose up -d
```

Esto levanta:

- **PostgreSQL 15** en el puerto `5432`.
- **SonarQube Community** en el puerto `9000` (opcional para análisis local).

2.6 Aplicar migraciones de base de datos

```
cd backend
alembic upgrade head
```

Este comando crea todas las tablas definidas en los modelos SQLAlchemy (users, profiles, calculation_history, user_preferences, favorite_conversions, sessions, sync_log).

---

3. Variables de entorno

Todas las variables de entorno se definen en `backend/.env`. **Nunca** commitee este archivo al repositorio.

Variable	Descripción	Valor de ejemplo
`DATABASE_URL`	URL de conexión a PostgreSQL con driver asyncpg	`postgresql+asyncpg://scicalc:secret@localhost:5432/scicalc_db`
`SECRET_KEY`	Clave secreta para firma de tokens JWT (mínimo 32 caracteres)	`mi-clave-super-secreta-de-al-menos-32-chars!`
`ALGORITHM`	Algoritmo de firma JWT	`HS256`
`ACCESS_TOKEN_EXPIRE_MINUTES`	Duración del token de acceso en minutos	`15`
`REFRESH_TOKEN_EXPIRE_DAYS`	Duración del token de refresco en días	`7`

> **Seguridad:** Use una `SECRET_KEY` de al menos 32 caracteres aleatorios. Genérela con: `python -c "import secrets; print(secrets.token_hex(32))"`.

---

4. Ejecutar el servidor

4.1 Inicio del servidor de desarrollo

```
cd backend
python run.py
```

El servidor queda disponible en: `http://localhost:8000`

Rutas principales:

- `http://localhost:8000/docs` — Documentación Swagger UI
- `http://localhost:8000/redoc` — Documentación ReDoc
- `http://localhost:8000/health` — Health check
- `http://localhost:8000/web/login` — Interfaz web de login
- `http://localhost:8000/web/register` — Interfaz web de registro
- `http://localhost:8000/web/calculator` — Interfaz web de la calculadora

4.2 Inicio con Uvicorn directamente

```
cd backend
uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload
```

El flag `--reload` recarga automáticamente al detectar cambios en el código (solo para desarrollo).

4.3 Producción con Nginx

En producción se recomienda colocar Nginx como reverse proxy frente a Uvicorn:

- Nginx escucha en el puerto `443` (HTTPS) y reenvía a `localhost:8000`.
- El certificado TLS debe estar configurado en Nginx.
- Usar `gunicorn` con workers `uvicorn.workers.UvicornWorker` para mayor estabilidad en producción.

---

5. Ejecución de pruebas unitarias

Las pruebas unitarias no requieren base de datos. Usan mocks para aislar la lógica de negocio.

```
cd backend
pytest tests/unit/ -v \\
  --cov=src \\
  --cov-report=term-missing \\
  --cov-report=xml:coverage.xml
```

Criterio de éxito: Las pruebas unitarias por sí solas generan cobertura aproximada del 42%. La cobertura total del 90% se alcanza al combinar con las pruebas de integración (ver sección 6).

Archivos de salida:

- `backend/coverage.xml` — Reporte de cobertura para SonarQube.

---

6. Ejecución de pruebas de integración

Las pruebas de integración requieren una instancia de PostgreSQL activa. Configure `DATABASE_URL` antes de ejecutarlas.

```
cd backend
pytest tests/unit/ tests/integration/ -v \\
  --cov=src \\
  --cov-report=term-missing \\
  --cov-report=xml:coverage.xml \\
  --cov-fail-under=90
```

Prerrequisito: La base de datos de prueba debe existir y tener las migraciones aplicadas:

```
alembic upgrade head
```

Criterio de éxito: Cobertura combinada mayor o igual al **90%** (`--cov-fail-under=90`). El pipeline CI fallará si no se alcanza este umbral.

---

7. Ejecución de pruebas E2E con Selenium

Las pruebas E2E levantan el servidor FastAPI en un hilo de fondo y controlan Chrome mediante Selenium WebDriver.

7.1 Prerrequisitos

- Google Chrome instalado en el sistema.
- `DATABASE_URL` apuntando a una base de datos PostgreSQL de prueba (puede ser la misma de integración o una dedicada).
- Migraciones aplicadas.

7.2 Comando de ejecución

```
cd backend
pytest tests/e2e/ -v -m e2e
```

7.3 Cobertura de pruebas E2E

ID	Archivo	Descripción
WEB-001	test_web_pages.py	Renderizado de `/web/login` con todos los elementos DOM requeridos
WEB-002	test_web_pages.py	Renderizado de `/web/register` con todos los elementos DOM requeridos
WEB-003	test_web_pages.py	Renderizado de `/web/calculator` con expression, precision, result, history-list
E2E-002a	test_auth_e2e.py	Registro de usuario nuevo → mensaje de éxito
E2E-002b	test_auth_e2e.py	Registro con email duplicado → mensaje de error
E2E-003a	test_auth_e2e.py	Login con credenciales válidas → token en sessionStorage
E2E-003b	test_auth_e2e.py	Login con contraseña incorrecta → sin token
E2E-004a	test_calculator_e2e.py	Aritmética básica `2 + 3 * 4 = 14`
E2E-004b	test_calculator_e2e.py	Trigonometría `sin(90)` en DEG = 1
E2E-004c	test_calculator_e2e.py	Expresión inválida → mensaje de error
E2E-005a	test_calculator_e2e.py	Historial muestra entradas después de calcular

ID	Archivo	Descripción
E2E-005b	test_calculator_e2e.py	Borrar entrada de historial → desaparece del DOM

7.4 Notas de ejecución en Windows

En Windows, el event loop de asyncpg requiere `SelectorEventLoop`. El archivo `conftest.py` aplica el parche automáticamente:

```
if sys.platform == "win32":
    asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
```

7.5 ChromeDriver

El paquete `webdriver-manager` descarga automáticamente la versión de ChromeDriver compatible con el Chrome instalado. No es necesario instalar ChromeDriver manualmente.

---

8. Análisis de calidad con SonarQube

8.1 Iniciar SonarQube con Docker

```
docker run -d \\
  --name sonarqube \\
  -p 9000:9000 \\
  sonarqube:community
```

Espere aproximadamente 60 segundos hasta que SonarQube esté disponible en `http://localhost:9000`.

Credenciales por defecto: `admin` / `admin` (cambiarlas en el primer acceso).

8.2 Crear proyecto en SonarQube

1. Acceda a `http://localhost:9000`.
2. Haga clic en **Create Project** → **Manually**.
3. Nombre del proyecto: `scicalcpro`, clave: `scicalcpro`.
4. Genere un **token de análisis** y guárdelo como `SONAR_TOKEN`.

8.3 Configurar sonar-project.properties

El archivo `backend/sonar-project.properties` debe contener:

```
sonar.projectKey=scicalcpro
sonar.projectName=SciCalc Pro
sonar.sources=src
sonar.tests=tests
sonar.python.coverage.reportPaths=coverage.xml
sonar.host.url=http://localhost:9000
```

8.4 Ejecutar el análisis

Genere primero el reporte de cobertura (sección 6), luego ejecute:

```
cd backend
sonar-scanner \\
  -Dsonar.token=$SONAR_TOKEN \\
  -Dsonar.host.url=http://localhost:9000
```

El informe estará disponible en `http://localhost:9000/dashboard?id=scicalcpro`.

---

9. Pipeline CI/CD con GitHub Actions

El pipeline está definido en `.github/workflows/ci.yml` y se activa en:

- Push a las ramas `main` o `develop`.
- Pull Requests hacia `main`.

9.1 Etapas del pipeline

Etapa	Job	Depende de	Descripción	Criterio de éxito
1	`lint`	—	Análisis estático con `ruff` sobre `src/`	Sin errores de lint
2	`unit-tests`	`lint`	Pruebas unitarias con cobertura parcial (~42%)	Todas las pruebas pasan
3	`integration-tests`	`lint`	Pruebas unitarias + integración con PostgreSQL 15	Cobertura >= 90%
4	`e2e-tests`	`integration-tests`	Pruebas E2E con Selenium + Chrome headless	Todas las pruebas pasan
5	`sonarqube`	`integration-tests`	Análisis SonarQube usando cobertura combinada	Condición: `SONAR_HOST_URL` configurado

9.2 Secrets y variables requeridos en GitHub

Configure estos valores en **Settings** → **Secrets and variables** → **Actions** del repositorio:

Nombre	Tipo	Descripción
`SONAR_TOKEN`	Secret	Token de autenticación de SonarQube
`SONAR_HOST_URL`	Variable (vars)	URL del servidor SonarQube (ej. `https://sonar.miempresa.com`)

> **Nota:** El job `sonarqube` se ejecuta condicionalmente: solo si `vars.SONAR_HOST_URL` está configurado. Esto evita fallos en forks que no tienen SonarQube.

9.3 Base de datos en CI

Los jobs `integration-tests` y `e2e-tests` usan el servicio Docker integrado de GitHub Actions para levantar PostgreSQL 15 Alpine:

```
services:
  postgres:
    image: postgres:15-alpine
    env:
      POSTGRES_DB: scicalc_test
      POSTGRES_USER: scicalc
      POSTGRES_PASSWORD: secret
```

Las variables de entorno del job apuntan automáticamente a este servicio:

```
DATABASE_URL: postgresql+asyncpg://scicalc:secret@localhost:5432/scicalc_test
```

9.4 Instalación de Chrome en CI

El job `e2e-tests` instala Google Chrome estable desde el repositorio oficial de Google:

```
wget -q -O - <https://dl.google.com/linux/linux_signing_key.pub> | sudo apt-key
add -
sudo sh -c 'echo "deb [arch=amd64] <http://dl.google.com/linux/chrome/deb/> sta
ble main" >> /etc/apt/sources.list.d/google-chrome.list'
sudo apt-get update -q && sudo apt-get install -y google-chrome-stable
```

---

10. Endpoints de la API

La documentación completa está disponible en `/docs` (Swagger UI) cuando el servidor está en ejecución.

Autenticación

Método	Ruta	Descripción
`POST`	`/api/v1/auth/register`	Registrar nuevo usuario
`POST`	`/api/v1/auth/login`	Iniciar sesión, obtener JWT
`POST`	`/api/v1/auth/refresh`	Renovar token de acceso
`POST`	`/api/v1/auth/logout`	Invalidar sesión

Usuarios

Método	Ruta	Descripción
`GET`	`/api/v1/users/me`	Obtener datos del usuario autenticado
`PUT`	`/api/v1/users/me`	Actualizar perfil
`DELETE`	`/api/v1/users/{user_id}`	Eliminar cuenta (soft delete)

Calculadora

Método	Ruta	Descripción
`POST`	`/api/v1/calculator/calculate`	Evaluar expresión matemática
`GET`	`/api/v1/calculator/history`	Obtener historial del usuario
`DELETE`	`/api/v1/calculator/history/{id}`	Eliminar entrada del historial

Conversión de unidades

Método	Ruta	Descripción
`POST`	`/api/v1/converter/convert`	Convertir entre unidades

Utilidades

Método	Ruta	Descripción
`GET`	`/health`	Health check del servidor

Interfaces web (Jinja2)

Método	Ruta	Descripción
`GET`	`/web/login`	Página de inicio de sesión
`GET`	`/web/register`	Página de registro
`GET`	`/web/calculator`	Interfaz de la calculadora

---

11. Arquitectura del sistema

11.1 Estilo arquitectónico: Hexagonal (Ports & Adapters)

SciCalc Pro sigue una arquitectura hexagonal para separar la lógica de negocio de los adaptadores externos (base de datos, HTTP, etc.).

```
src/
├── domain/          # Entidades, value objects, reglas de negocio puras
├── application/     # Casos de uso, servicios de aplicación
├── infrastructure/  # Adaptadores: repositorios SQLAlchemy, modelos DB
├── api/             # Adaptadores de entrada: routers FastAPI
├── web/             # Adaptadores de entrada: vistas Jinja2
└── main.py          # Punto de entrada, composición de la aplicación
```

11.2 Capas

Capa	Tecnología	Responsabilidad
Presentación web	Jinja2 + HTML/CSS/JS	Renderizado server-side de páginas web
API REST	FastAPI 0.111	Endpoints JSON, validación con Pydantic
Aplicación	Python puro	Casos de uso, orquestación
Dominio	Python puro	Entidades, reglas de negocio
Persistencia	SQLAlchemy 2.0 + asyncpg	ORM asíncrono sobre PostgreSQL
Autenticación	PyJWT + bcrypt	Tokens JWT, hashing de contraseñas
Cálculo	mpmath	Aritmética de precisión arbitraria

11.3 Flujo de una solicitud de cálculo

```
Navegador
  → POST /api/v1/calculator/calculate
    → Router FastAPI (valida JWT, deserializa body)
      → CalculatorService (caso de uso)
        → ExpressionValidator (regex RNF-006)
        → MathEngine (mpmath, evaluación segura)
        → HistoryRepository (SQLAlchemy async → PostgreSQL)
      ← resultado + entrada guardada en historial
    ← JSON { result, expression, precision }
  ← Actualiza DOM (result, history-list)
```

11.4 Seguridad

- **Autenticación:** JWT HS256, tokens de acceso de 15 min + refresh de 7 días.

- **Contraseñas:** bcrypt con factor de coste 12.
- **Validación de expresiones:** Regex lista blanca (RNF-006) antes de evaluar. Solo permite caracteres matemáticos conocidos.
- **Soft delete:** Todas las entidades incluyen `is_deleted` para eliminación lógica sin pérdida de datos.
- **Variables sensibles:** Cargadas desde variables de entorno, nunca hardcodeadas.